Practical - 1

1. Call by value

```
#include <stdio.h>

int sum(int a, int b)
{
    return a + b;
}
int main()
{
    printf("Call by value\n");
    int a, b, s;
    printf("Enter A : ");
    scanf("%d", &a);
    printf("Enter B : ");
    scanf("%d", &b);
    s = sum(a, b);
    printf("Sum of A & B = %d", s);
    return 0;
}
```

```
HitenRathod135@DataStructure >gcc Practical_1.c

HitenRathod135@DataStructure >a.exe
Call by value
Enter A : 12
Enter B : 15
Sum of A & B = 27
HitenRathod135@DataStructure >
```

Practical – 1

2. Call by Reference

```
#include <stdio.h>


int sum(int *a, int *b)
{
    return *a + *b;
}
int main()
{
    printf("Call by Rerence\n");
    int a, b, s;
    printf("Enter A : ");
    scanf("%d", &a);
    printf("Enter B : ");
    scanf("%d", &b);
    s = sum(&a, &b);
    printf("Sum of A & B = %d", s);
    return 0;
}
```

```
HitenRathod135@DataStructure >gcc Practical_1_2.c

HitenRathod135@DataStructure >a.exe
Call by Rerence
Enter A : 22
Enter B : 33
Sum of A & B = 55
HitenRathod135@DataStructure >
```

Practical – 2

Malloc() and calloc() :

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
   printf("Program of malloc(), calloc() and free()\n");
   // syntax for malloc()
   // ptr  = (int *) malloc(total_size_of_array);
   int *ptr, n;
   printf("1. malloc()\n");
   printf("Enter size of Array : ");
   scanf("%d", &n);
   ptr = (int *)malloc(n * sizeof(int));
   for (int i = 0; i < n; i++)
   {
      scanf("%d", &ptr[i]);
   }
   for (int i = 0; i < n; i++)
   {
      printf("%d  ", ptr[i]);
   }
   free(ptr);
   // syntax for calloc()
   // ptr = (int *) calloc(n , sizeof(data_type));
   printf("\n2. calloc()");
   printf("\nEnter size of Array : ");
   scanf("%d", &n);
   ptr = (int *)calloc(n, sizeof(int));
   for (int i = 0; i < n; i++)
   {
      scanf("%d", &ptr[i]);
   }
   for (int i = 0; i < n; i++)
   {
```

```
        printf("%d  ", ptr[i]);
    }
    // use of free for free that memory
    free(ptr);
    return 0;
}
```

```
HitenRathod135@DataStructure >gcc practical_2.c

HitenRathod135@DataStructure >a.exe
Program of malloc(), callo() and free()
1. malloc()
Enter size of Array : 3
11
12
13
11  12  13
2. calloc()
Enter size of Array : 4
111
222
333
444
111  222  333  444
HitenRathod135@DataStructure >
```

Practical - 3

Stack using Array :

```
#include <stdio.h>
```

```
#include <stdlib.h>
#define size 5
int top = -1;

void DISPLAY(int *A)
{
   if (top == -1)
   {
      printf("Stack is Empty");
   }
   for (int i = 0; i <= top; i++)
   {
      printf("%d  ", A[i]);
   }
}




void PUSH(int *A)
{

   int data;
   if (top == 4)
   {
      printf("Stack is Full");
   }
   else
   {
      printf("Enter the Data : ");
      scanf("%d", &data);
      top = top + 1;
      A[top] = data;
      printf("%d Inserted\n", data);
```

```
    }
}

void POP(int *A)
{
    if (top == -1)
    {
        printf("Stack is Empty");
    }
    else
    {
        printf("%d Poped\n", A[top]);
        top = top - 1;
    }
}

void PEEP(int *A)
{
    int p;
    printf("Enter Index of Element : ");
    scanf("%d", &p);
    if (top == -1)
    {
        printf("Stack is Empty");
    }
    else if (p > top)
    {
        printf("Invalid Position");
    }
    else
    {
        printf("Element at Position %d is : %d", p, A[p]);
    }
```

```
}

void CHANGE(int *A)
{
   int CheckData, NewData, Ind = 0;
   printf("\nEnter the Number you want to change : ");
   scanf("%d", &CheckData);
   if (top == -1)
   {
      printf("Element Not found\n");
   }
   else
   {
      while (A[Ind] != CheckData && Ind <= top)
      {
         Ind++;
      }
      if (Ind == top + 1)
      {
         printf("Element Not found\n");
      }
      else
      {
         printf("Enter New number : ");
         scanf("%d", &NewData);
         A[Ind] = NewData;
         printf("Element changed\n");
      }
   }
}
int main()
{
   int array[size], choice;
```

```
    printf("1. PUSH\n2. POP\n3. PEEP\n4.CHANGE\n5.DISPLAY\n6. Exit");
    while (1)
    {
       printf("\nEnter Choice : ");
       scanf("%d", &choice);
       switch (choice)
       {


          case 1:
          PUSH(array);
          break;
       case 2:
          POP(array);
          break;
       case 3:
          PEEP(array);
          break;
       case 4:
          CHANGE(array);
          break;
       case 5:
          DISPLAY(array);
          break;
       case 6:
          exit(0);
       default:
          printf("Invalid Choice");
       }
    }
    return 0;
}
```

```
HitenRathod135@DataStructure >gcc Practical_3.c

HitenRathod135@DataStructure >a.exe
1. PUSH
2. POP
3. PEEP
4.CHANGE
5.DISPLAY
6. Exit
Enter Choice : 1
Enter the Data : 11
11 Inserted

Enter Choice : 1
Enter the Data : 134
134 Inserted

Enter Choice : 1
Enter the Data : 432
432 Inserted

Enter Choice : 5
11  134  432
Enter Choice : 1
Enter the Data : 5453
5453 Inserted

Enter Choice : 5
11  134  432  5453
Enter Choice : 2
5453 Poped

Enter Choice : 2
432 Poped

Enter Choice : 5
11  134
Enter Choice : 1
Enter the Data : -432
-432 Inserted

Enter Choice : 5
11  134  -432
Enter Choice : 6
```

Practical – 4

Queue using Array :

#include <stdio.h>

```
#include <string.h>

#include <stdlib.h>

int top = -1, i = 0, j = 0;

void push(char *stack, char c)
{
   top = top + 1;
   stack[top] = c;
}

char pop(char *stack)
{
   char ret = stack[top];
   top = top - 1;
   return ret;
}


int isoperator(char c)
{
   if (c == '+' || c == '-' || c == '*' || c == '/')
   {
      return 1;
   }
   return 0;
```

```
}
int precidence(char c)
{
   if (c == '+' || c == '-')
   {
      return 1;
   }
   else if (c == '*' || c == '/')
   {
      return 2;
   }
   return 0;
}

int main()
{
   char exp[20], *stack, *Postfix;

   printf("Enter Expression : ");
   gets(exp);

   stack = (char *)malloc((strlen(exp) + 1) * sizeof(char));
   Postfix = (char *)malloc((strlen(exp) + 1) * sizeof(char));

   Postfix[strlen(exp)] = '\0';

   while (exp[i] != '\0')
   {
```

```
    if (!isoperator(exp[i]))
    {
       Postfix[j] = exp[i];
       i++;
       j++;
    }

 else
    {
       if (precidence(exp[i]) > precidence(stack[top]))
       {
          push(stack, exp[i]);
          i++;
       }
       else
       {
          Postfix[j] = pop(stack);
          j++;
       }
    }
 }

while (top != -1)
 {
    Postfix[j] = stack[top];
    j++;
    top--;
 }
```

```
    printf("PostFix is : %s", Postfix);

    return 0;
}
```

```
HitenRathod135@DataStructure >gcc Practical_4.c

HitenRathod135@DataStructure >a.exe
Enter Expression : a+b*g/h*d-t
PostFix is : abg*h/d*+t-
HitenRathod135@DataStructure >
```

Practical – 5

Infix to Postfix :

```
#include <stdio.h>
#include <stdlib.h>
#define size 5

int front = -1, rear = -1;

void display(int *A)
{
   if (rear == -1 && front == -1)
   {
      printf("Queue is Empty");
   }
   else
   {
      for (int i = rear; i <= front; i++)
      {
         printf("%d   ", A[i]);
      }
   }
}

void Insert(int *A)
{
   int data;
   if (front == 4)
   {
      printf("\nQueue is Full\n");
   }
```

```
    else if (rear == -1 && front == -1)
    {
        printf("\nEnter data : ");
        scanf("%d", &data);

        front++;
        rear++;

        A[front] = data;
        printf("%d Inserted\n", data);
    }
    else
    {
        printf("\nEnter data : ");
        scanf("%d", &data);

        front++;

        A[front] = data;
        printf("%d Inserted\n", data);
    }
}

int delete(int *A)
{
    int free;
    if (front == rear)
    {
```

```
        free = A[front];
        front = -1;
        rear = -1;
        printf("%d removed\n", free);
        return free;
    }
    free = A[rear];
    rear++;
    printf("%d removed\n", free);
    return free;
}

int main()
{
    int array[size], choice;
    printf("1. Insert\n2. Delete\n3. Display\n4. Exit");
    while (1)
    {

        printf("\nEnter choice : ");
        scanf("%d", &choice);
        switch (choice)
        {
        case 1:
            Insert(array);
            break;
        case 2:
            delete (array);
```

```
            break;
        case 3:
            display(array);
            break;
        case 4:
            exit(0);
            break;

        default:
            printf("Invalid choice");
            break;
        }
    }
    return 0;
}
```

```
HitenRathod135@DataStructure >gcc Practical_5.c

HitenRathod135@DataStructure >a.exe
1. Insert
2. Delete
3. Display
4. Exit
Enter choice : 1

Enter data : 12
12 Inserted

Enter choice : 1

Enter data : 234
234 Inserted

Enter choice : 1

Enter data : 543
543 Inserted

Enter choice : 3
12   234   543
Enter choice : 3
12   234   543
Enter choice : 2
12 removed

Enter choice : 3
234   543
Enter choice : 2
234 removed

Enter choice : 3
543
Enter choice : 2
543 removed

Enter choice : 3
Queue is Empty
Enter choice : 4

HitenRathod135@DataStructure >
```

Practical – 6

Implement circular Queue using Array :

```
#include <stdio.h>
#include <stdlib.h>
#define size 5
int front = -1, rear = -1;
void Insertion(int *A)
{
   int data;
   printf("Enter data : ");
   scanf("%d", &data);
   if (rear == -1 && front == -1)
   {
      front++;
      rear++;
      A[front] = data;
      printf("%d Inserted\n", data);
   }
   else if (rear <= front && front != size - 1)
   {
      front++;
      A[front] = data;
      printf("%d Inserted\n", data);
   }
   else if (front == size - 1)
   {
      front = 0;
      if (front == rear)
      {
         printf("Circular Queue is full\n");
```

```
            front = size - 1;
        }
        else
        {
            A[front] = data;
            printf("%d Inserted\n", data);
        }
    }
    else if (front < rear)
    {
        front++;
        if (front == rear)
        {
            front--;
            printf("Circular Queue is full\n");
        }
        else
        {
            A[front] = data;
            printf("%d Inserted\n", data);
        }
    }
}

int Deletion(int *A)
{
    int free;
    if (front == -1 && rear == -1)
```

```
    {
        printf("Circular Queue is Empty\n");
        return 0;
    }
    else if (front == rear)
    {
        free = A[rear];
        front = -1;
        rear = -1;
        printf("%d removed\n", free);
        return free;
    }
    else if (rear == size - 1)
    {
        free = A[rear];
        rear = 0;
        printf("%d removed\n", free);
        return free;
    }
    else
    {
        free = A[rear];
        rear++;
        printf("%d removed\n", free);
        return free;
    }
}
```

```
void Display(int *A)
{
   if (front == -1 && rear == -1)
   {
      printf("Circular Queue is Empty\n");
   }
   else if (rear <= front)
   {
      for (int i = rear; i <= front; i++)
      {
         printf("%d  ", A[i]);
      }
   }
   else
   {
      for (int i = rear; i < size; i++)
      {
         printf("%d  ", A[i]);
      }
      for (int i = 0; i <= front; i++)
      {
         printf("%d  ", A[i]);
      }
   }
}
int main()
{
   int Array[size], choice;
```

```
    printf("1. Insertion\n2. deletion\n3. Display\n4. Exit\n");
    while (1)
    {
        printf("\nEnter Choice : ");
        scanf("%d", &choice);
        switch (choice)
        {
        case 1:
            Insertion(Array);
            break;
        case 2:
            Deletion(Array);
            break;
        case 3:
            Display(Array);
            break;
        case 4:
            exit(0);
            break;

        default:
            printf("Invalid choice");
        }
    }

    return 0;
}
```

```
HitenRathod135@DataStructure >a.exe
1. Insertion
2. deletion
3. Display
4. Exit

Enter Choice : 1
Enter data : 11
11 Inserted

Enter Choice : 1
Enter data : 12
12 Inserted

Enter Choice : 1
Enter data : 13
13 Inserted

Enter Choice : 1
Enter data : 14
14 Inserted

Enter Choice : 1
Enter data : 15
15 Inserted

Enter Choice : 1
Enter data : 123
Circular Queue is full

Enter Choice : 2
11 removed

Enter Choice : 1
Enter data : 123
123 Inserted

Enter Choice : 3
12  13  14  15  123
Enter Choice : 2
12 removed

Enter Choice : 3
13  14  15  123
Enter Choice : 4

HitenRathod135@DataStructure >_
```

Practical – 7

Singly linked list :

```
#include <stdio.h>
#include <stdlib.h>
struct link
{
   int data;
   struct link *next;
};
struct link *Insert_First(struct link *First)
{
   int n;
   printf("Enter the Data : ");
   scanf("%d", &n);
   struct link *ptr = (struct link *)malloc(sizeof(struct link));
   ptr->data = n;
   ptr->next = First;
   return ptr;
}

struct link *Insert_Last(struct link *First)
{
   int n;
   printf("Enter the Data : ");
   scanf("%d", &n);
   struct link *p = (struct link *)malloc(sizeof(struct link));
   struct link *ptr = First;
   while (ptr->next != NULL)
   {
      ptr = ptr->next;
```

```
    }
    ptr->next = p;
    p->data = n;
    p->next = NULL;
    return p;
}

struct link *Insert_At_Index(struct link *First)
{
    int n, index;
    printf("Enter Index : ");
    scanf("%d", &index);
    printf("Enter the Data : ");
    scanf("%d", &n);
    int i = 0;
    struct link *ptr = (struct link *)malloc(sizeof(struct link));
    struct link *p = First;
    while (i != index - 1)
    {
        p = p->next;
        i = i + 1;
    }
    ptr->next = p->next;
    p->next = ptr;
    ptr->data = n;
    return ptr;
}
```

```
struct link *Delete_First(struct link *First)
{
   struct link *freeptr = First;
   First = First->next;
   free(freeptr);
   return First;
}

void Delete_Before_Value(struct link *First, int info)
{
   struct link *t, *t2, *t3;
   t = First;
   if (info == First->data)
   {
      printf("node cannot be deleted\n");
   }
   else
   {
      if (info == First->next->data)
      {
         t3 = First;
         First = First->next;
         free(t3);
      }
      else
      {
         while (t->next->next->data != info && t->next->next !=
NULL)
```

```
            {
               t = t->next;
            }
            if (t->next->next->data == info)
            {
               t2 = t->next;
               t->next = t->next->next;
               free(t2);
            }
         }
      }
   }

void Delete_After_Value(struct link *First)
{
   struct link *ptr, *ptr1;
   int loc, i;
   scanf("%d", &loc);
   ptr = First;
   for (i = 0; i < loc; i++)
   {
      ptr1 = ptr;
      ptr = ptr->next;

      if (ptr == NULL)
      {
         printf("\nThere are less than %d elements in the list..
\n", loc);
```

```
            return;
        }
    }
    ptr1->next = ptr->next;
    free(ptr);
    printf("\nDeleted %d node ", loc);
}

void Display(struct link *First)
{
    struct link *free;
    free = First;
    if (First == NULL)
    {
        printf("Stack is Empty\n");
    }
    else
    {
        while (free != NULL)
        {
            printf("%d  ", free->data);
            free = free->next;
        }
    }
}

int main()
{
```

```
    int choice, info;
    struct link *First = NULL;
    printf("1. Insert First\n2. Insert Last\n3. Insert At Index\n");
    printf("4. Delete First\n5. Delete Before Value\n6. Delete
After Value\n7. Exit\n8. Display\n");
    while (1)
    {
        printf("\nEnter choice : ");
        scanf("%d", &choice);
        switch (choice)
        {
        case 1:
            First = Insert_First(First);
            break;
        case 2:
            Insert_Last(First);
            break;
        case 3:
            Insert_At_Index(First);
            break;
        case 4:
            First = Delete_First(First);
            break;
        case 5:
            printf("Enter value : ");
            scanf("%d", &info);
            Delete_Before_Value(First, info);
            break;
```

```
        case 6:
            Delete_After_Value(First);
            break;
        case 8:
            Display(First);
            break;
        case 7:
            exit(0);
            break;
        default:
            printf("\nIndvalid Choice");
        }
    }

    return 0;
}
```

```
HitenRathod135@DataStructure >a.exe
1. Insert First
2. Insert Last
3. Insert At Index
4. Delete First
5. Delete Before Value
6. Delete After Value
7. Exit
8. Display

Enter choice : 1
Enter the Data : 11

Enter choice : 1
Enter the Data : 22

Enter choice : 1
Enter the Data : 33

Enter choice : 1
Enter the Data : 44

Enter choice : 8
44  33  22  11
Enter choice : 4

Enter choice : 8
33  22  11
Enter choice : 2
Enter the Data : 21

Enter choice : 8
33  22  11  21
Enter choice : 1
Enter the Data : 555

Enter choice : 8
555  33  22  11  21
Enter choice : 76

Indvalid Choice
Enter choice : 7

HitenRathod135@DataStructure >
```

Practical – 8

Stack using linked list :

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
   int Data;
   struct Node *Next;
};
struct Node *Insert(struct Node *First)
{
   struct Node *ptr;
   int n;
   printf("Enter the Data : ");
   scanf("%d", &n);
   ptr = (struct Node *)malloc(sizeof(struct Node));
   ptr->Data = n;
   if (First == NULL)
   {
      ptr->Next = NULL;
      printf("%d Inserted\n", n);
   }
   else
   {
      printf("%d Inserted\n", n);
      ptr->Next = First;
   }
   return ptr;
}
```

```
struct Node *Delete(struct Node *First)
{
   if (First == NULL)
   {
      printf("Stack is Empty\n");
   }
   else
   {
      struct Node *Free;
      Free = First;
      First = First->Next;
      printf("%d Deleted\n", Free->Data);
      free(Free);
      return First;
   }
}

void Display(struct Node *First)
{
   struct Node *free;
   free = First;
   if (First == NULL)
   {
      printf("Stack is Empty\n");
   }
   else
   {
```

```
        while (free != NULL)
        {
            printf("%d  ", free->Data);
            free = free->Next;
        }
    }
}

int main()
{
    struct Node *First = NULL;
    int choice;
    printf("1. Insert\n2. Delete\n3. Display\n4. Exit");
    while (1)
    {
        printf("\nEnter choice : ");
        scanf("%d", &choice);
        switch (choice)
        {
        case 1:
            First = Insert(First);
            break;
        case 2:
            First = Delete(First);
            break;
        case 3:
            Display(First);
            break;
```

```
        case 4:
            exit(0);
            break;

        default:
            printf("Invalid Choice");
        }
    }

    return 0;
}
```

```
HitenRathod135@DataStructure >a.exe
1. Insert
2. Delete
3. Display
4. Exit
Enter choice : 1
Enter the Data : 12
12 Inserted

Enter choice : 3
12
Enter choice : 1
Enter the Data : 22
22 Inserted

Enter choice : 3
22  12
Enter choice : 1
Enter the Data : 32
32 Inserted

Enter choice : 1
Enter the Data : 435
435 Inserted

Enter choice : 3
435  32  22  12
Enter choice : 2
435 Deleted

Enter choice : 3
32  22  12
Enter choice : 2
32 Deleted

Enter choice : 3
22  12
Enter choice : 4

HitenRathod135@DataStructure >
```

Practical – 9

Queue using linked list :

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
   int Data;
   struct Node *Next;
};
struct Node *Insert(struct Node *First)
{
   struct Node *ptr;
   int n;
   printf("Enter the Data : ");
   scanf("%d", &n);
   ptr = (struct Node *)malloc(sizeof(struct Node));
   ptr->Data = n;
   if (First == NULL)
   {
      ptr->Next = NULL;
      printf("%d Inserted\n", n);
   }
   else
   {
      printf("%d Inserted\n", n);
      ptr->Next = First;
   }
```

```
    return ptr;
}

struct Node *Delete(struct Node *First)
{
    struct Node *after = First, *prev;
    if (First == NULL)
    {
        printf("Stack is Empty\n");
        return NULL;
    }
    else if (after->Next == NULL)
    {
        printf("%d removed\n", after->Data);
        return NULL;
    }
    else
    {

        while (after->Next != NULL)
        {
            prev = after;
            after = after->Next;
        }
        prev->Next = NULL;
        printf("%d removed\n", after->Data);
        free(after);
        return First;
```

```
    }
}

void Display(struct Node *First)
{
    struct Node *free;
    free = First;
    if (First == NULL)
    {
        printf("Stack is Empty\n");
    }
    else
    {
        while (free != NULL)
        {
            printf("%d  ", free->Data);
            free = free->Next;
        }
    }
}

int main()
{
    struct Node *First = NULL;
    int choice;
    printf("1. Insert\n2. Delete\n3. Display\n4. Exit");
    while (1)
    {
```

```
        printf("\nEnter choice : ");
        scanf("%d", &choice);
        switch (choice)
        {
        case 1:
            First = Insert(First);
            break;
        case 2:
            First = Delete(First);
            break;
        case 3:
            Display(First);
            break;

        case 4:
            exit(0);
            break;

        default:
            printf("Invalid Choice");
        }
    }

    return 0;
}
```

```
HitenRathod135@DataStructure >a.exe
1. Insert
2. Delete
3. Display
4. Exit
Enter choice : 1
Enter the Data : 11
11 Inserted

Enter choice : 1
Enter the Data : 22
22 Inserted

Enter choice : 1
Enter the Data : 33
33 Inserted

Enter choice : 1
Enter the Data : 44
44 Inserted

Enter choice : 1
Enter the Data : 55
55 Inserted

Enter choice : 3
55  44  33  22  11
Enter choice : 2
11 removed

Enter choice : 3
55  44  33  22
Enter choice : 2
22 removed

Enter choice : 3
55  44  33
Enter choice : 2
33 removed

Enter choice : 3
55  44
Enter choice : 4

HitenRathod135@DataStructure >
```

Practical – 10

Doubly linked list :

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *next;
    struct Node *prev;
};

void insert_at_begin(struct Node **head)
{
    int d;
    printf("Enter data : ");
    scanf("%d", &d);

   struct Node *newNode = (struct Node *)malloc(sizeof(struct
Node));
    newNode->data = d;
    newNode->next = *head;
    newNode->prev = NULL;

    if (*head != NULL)
        (*head)->prev = newNode;
    *head = newNode;
}
```

```
void insert_at_end(struct Node **head)
{
   int d;
   printf("Enter data : ");
   scanf("%d", &d);
   struct Node *newNode = (struct Node
*)malloc(sizeof(struct Node));
   newNode->data = d;
   newNode->next = NULL;
   if (*head == NULL)
   {
      *head = newNode;
      newNode->prev = NULL;
      return;
   }
  struct Node *temp = *head;
   while (temp->next != NULL)
      temp = temp->next;

   temp->next = newNode;
   newNode->prev = temp;
}
int calcSize(struct Node *node)
{
   int size = 0;

   while (node != NULL)
   {
```

```
        node = node->next;
        size++;
    }
    return size;
}
void insert_before_position(struct Node **head)
{
    int d, pos;
    printf("\nEnter data : ");
    scanf("%d", &d);
    printf("\nEnter Position : ");
    scanf("%d", &pos);
    int size = calcSize(*head);

    if (pos < 0 || size < (pos - 1))
    {
        printf("Can't insert, %d is not a valid position\n", (pos -
1));
        return;
    }

    if (pos == 0)
        insert_at_begin(head);

    else if (pos == size)
        insert_at_end(head);

    else
```

```
    {
        struct Node *temp = *head;
        struct Node *newNode = (struct Node
*)malloc(sizeof(struct Node));
        newNode->data = d;
        newNode->next = NULL;
        while (--pos - 1)
            temp = temp->next;
        struct Node *temp2 = temp->next;
        newNode->next = temp->next;
        newNode->prev = temp;
        temp->next = newNode;
        temp2->prev = newNode;
    }
}
void display(struct Node *node)
{
  struct Node *ptr;
    if (node == NULL)
    {
        printf("\nEmpty List\n");
    }
    else
    {
        ptr = node;
        while (ptr != NULL)
        {
            printf("%d\n", ptr->data);
```

```
            ptr = ptr->next;
        }
    }
}
void delete_last_node(struct Node *head)
{
    if (head->next == NULL)
    {
        free(head);
        head = NULL;
    }
    struct Node *extra = head, *prev_node;
    while (extra->next != NULL)
    {
        prev_node = extra;
        extra = extra->next;
    }
    free(extra); // t will delete the last node
    prev_node->next = NULL;
}
void main()
{
    struct Node *head = NULL;
    int choice;
    printf("1. insert_at_begin\n2. insert_at_end\n");
    printf("3. insert_before_position\n4. delete_last_node\n5.
Exit\n6. Display\n");
    while (1)
```

```
{
    printf("\nEnter choice : ");
    scanf("%d", &choice);
    switch (choice)
    {
    case 1:
        insert_at_begin(&head);
        break;
    case 2:
        insert_at_end(&head);
        break;
    case 3:
        insert_before_position(&head);
        break;
    case 4:
        delete_last_node(head);
        break;
    case 5:
        exit(0);
        break;
    case 6:
        display(head);
        break;

    default:
        printf("Invalid Choice");
        break;
    }
```

```
    }
}
```

```
HitenRathod135@DataStructure >a.exe
1. insert_at_begin
2. insert_at_end
3. insert_before_position
4. delete_last_node
5. Exit
6. Display

Enter choice : 1
Enter data : 11

Enter choice : 6
11

Enter choice : 1
Enter data : 22

Enter choice : 6
22
11

Enter choice : 1
Enter data : 555

Enter choice : 6
555
22
11

Enter choice : 2
Enter data : 444

Enter choice : 6
555
22
11
444

Enter choice : 3

Enter data : 11
```

```
Enter Position : 22
Can't insert, 21 is not a valid position

Enter choice : 4

Enter choice : 6
555
22
11

Enter choice : 3

Enter data : 321

Enter Position : 1

HitenRathod135@DataStructure >
```

Practical – 11

Circular linked list :

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *next;
};
struct Node *addToEmpty(struct Node *last, int data)
{
   if (last != NULL)
      return last;
   struct Node *temp = (struct Node *)malloc(sizeof(struct
Node));

   temp->data = data;
   last = temp;

   last->next = last;
   return last;
}

struct Node *Insert_End(struct Node *last)
{
   int n;
   printf("\nEnter data : ");
   scanf("%d", &n);
```

```
    if (last == NULL)
        return addToEmpty(last, n);

    struct Node *temp = (struct Node *)malloc(sizeof(struct
Node));

    temp->data = n;
    temp->next = last->next;
    last->next = temp;
    last = temp;

    return last;
}

struct Node *Insert_After_Position(struct Node *last)
{
    int data, item;
    printf("\nEnter the Data : ");
    scanf("%d", &item);
    printf("After which Data you want to enter : ");
    scanf("%d", &data);
    if (last == NULL)
        return NULL;

    struct Node *temp, *p;
    p = last->next;

    do
```

```
    {
        if (p->data == item)
        {
            temp = (struct Node *)malloc(sizeof(struct Node));
            temp->data = data;
            temp->next = p->next;
            p->next = temp;
            if (p == last)
                last = temp;
            return last;
        }
        p = p->next;
    } while (p != last->next);

    printf(" not present in the list.\n");
    return last;
}

void Display(struct Node *last)
{
    struct Node *p;

    if (last == NULL)
    {
        printf("List is empty.\n");
        return;
    }
```

```
    p = last->next;

    do
    {
        printf("%d  ", p->data);
        p = p->next;
    } while (p != last->next);
}

void Delete_First(struct Node *last)
{
    struct Node *p, *temp;
    p = last;
    while (p->next != last)
    {
        p = p->next;
    }
    temp = last;
    last = last->next;
    p->next = last;
    free(temp);
}

void Delete_After_Position(struct Node *last)
{
    struct Node *ptr, *ptr1;
    int loc, i;
    scanf("%d", &loc);
```

```
    ptr = last;
    for (i = 0; i < loc; i++)
    {
        ptr1 = ptr;
        ptr = ptr->next;

        if (ptr == NULL)
        {
            printf("\nThere are less than %d elements in the list..
\n", loc);
            return;
        }
    }
    ptr1->next = ptr->next;
    free(ptr);
    printf("\nDeleted %d node ", loc);
}

int main()
{
    struct Node *last = NULL;
    int choice;
    printf("1. Insert_End\n2. Insert_After_Position\n3.
Delete_First");
    printf("\n4. Delete_After_Position\n5. Display\n6. Exit\n");
    while (1)
    {
        printf("\nEnter choice : ");
        scanf("%d", &choice);
```

```
        switch (choice)
        {
        case 1:
            last = Insert_End(last);
            break;
        case 2:
            last = Insert_After_Position(last);
            break;
        case 3:
            Delete_First(last);
            break;
        case 4:
            Delete_After_Position(last);
            break;
        case 5:
            Display(last);
            break;
        case 6:
            exit(0);
            break;

        default:
            printf("Invalid Choice");
        }
    }

    return 0;
}
```

```
HitenRathod135@DataStructure >a.exe
1. Insert_End
2. Insert_After_Position
3. Delete_First
4. Delete_After_Position
5. Display
6. Exit

Enter choice : 1

Enter data : 11

Enter choice : 5
11
Enter choice : 1

Enter data : 323

Enter choice : 5
11  323
Enter choice : 1

Enter data : 234

Enter choice : 5
11  323  234
Enter choice : 1

Enter data : 765

Enter choice : 5
11  323  234  765
Enter choice : 4
3

Deleted 3 node
Enter choice : 5
11  323  765
Enter choice : 1

Enter data : -777

Enter choice : 5
11  323  765  -777
Enter choice : 6
```

Practical – 12

Bubble sort :

```
#include <stdio.h>
void TraversalArray(int *array, int size)
{
   for (int i = 0; i < size; i++)
   {
      printf("%d\t", array[i]);
   }
}

void BubbleSort(int *array, int size)
{
   int free;
   for (int i = 0; i < size-1; i++)
   {
      for (int j = 0; j < size - i; j++)
      {
         if (array[j] > array[j + 1])
         {
            free = array[j + 1];
            array[j + 1] = array[j];
            array[j] = free;
         }
      }
      printf("\n");
      TraversalArray(array, size);
   }
```

```
}
int main()
{
    int a[] = {3, 34, 0, 12, 20, -54, 323, 43, -123};
    TraversalArray(a, 9);
    BubbleSort(a, 9);
    // printf("\n");
    // TraversalArray(a, 5);
    return 0;
}
```

```
HitenRathod135@DataStructure >gcc Practical_12.c

HitenRathod135@DataStructure >a.exe
3       34      0       12      20      -54     323     43      -123
3       0       12      20      -54     34      43      -123    323
0       3       12      -54     20      34      -123    43      323
0       3       -54     12      20      -123    34      43      323
0       -54     3       12      -123    20      34      43      323
-54     0       3       -123    12      20      34      43      323
-54     0       -123    3       12      20      34      43      323
-54     -123    0       3       12      20      34      43      323
-123    -54     0       3       12      20      34      43      323
HitenRathod135@DataStructure >
```

Practical – 13

Quick sort :

```
}

int SetIndexOfFirst(int *array, int startindex, int size)
{
   int left = startindex, right = size, pivot = startindex, free;
   while (left != right)
   {
      if (pivot < right)
      {
         while (pivot != right)
         {
            if (array[pivot] >= array[right])
            {
               free = array[pivot];
               array[pivot] = array[right];
               array[right] = free;
               pivot = right;
            }
            else
            {
              right--;
            }
         }
      }
      else if (pivot > left)
      {
```

```
            while (pivot != left)
            {
               if (array[pivot] < array[left])
               {
                  free = array[pivot];
                  array[pivot] = array[left];
                  array[left] = free;
                  pivot = left;
               }
               else
               {
                  left++;
               }
            }
         }
      }
      return pivot;
}

void QuickSort(int *array, int startindex, int size)
{
   int IndexOfPivot;
   if (startindex < size)
   {
      IndexOfPivot = SetIndexOfFirst(array, startindex, size);
      QuickSort(array, startindex, IndexOfPivot - 1);
      QuickSort(array, IndexOfPivot + 1, size);
   }
```

```
}
int main()
{
    int array[] = {9, 5, 18, 22, 0, 5, 16, 555, 444, -342}, size;
    size = sizeof(array) / sizeof(array[0]);
    TraversalOfArray(array, size);
    QuickSort(array, 0, size - 1);
    printf("\n");
    TraversalOfArray(array, size);
    return 0;
}
```

```
HitenRathod135@DataStructure >gcc Practical_13.c

HitenRathod135@DataStructure >a.exe
9  5  18  22  0  5  16  555  444  -342
-342  0  5  5  9  16  18  22  444  555
HitenRathod135@DataStructure >
```

Practical – 14

Merge sort :

```
#include <stdio.h>


void TraversalArray(int *array, int size)
{
   for (int I = 0; I < size; i++)
   {
      printf(“%d\t”, array[i]);
   }
}


void MergeArray(int *array, int low, int mid, int high, int
ARRAYSIZE)
{
   int I = low, k = low, j = mid + 1, Newarray[ARRAYSIZE];


   while (I <= mid && j <= high)
   {
      if (array[i] < array[j])
      {
         Newarray[k] = array[i];
         i++;
         k++;
      }
      else
      {
         Newarray[k] = array[j];
```

```
            j++;
            k++;
        }
    }
    while (l <= mid)
    {
        Newarray[k] = array[i];
        k++;
        i++;
    }
    while (j <= high)
    {
        Newarray[k] = array[j];
        k++;
        j++;
    }
    for (int l = low; l <= high; i++)
    {
        array[i] = Newarray[i];
    }
}

void MergeSort(int *array, int low, int high, int ARRAYSIZE)
{
    int mid;
    if (low < high)
    {
        mid = (low + high) / 2;
```

```
        MergeSort(array, low, mid, ARRAYSIZE);
        MergeSort(array, 1 + mid, high, ARRAYSIZE);
        MergeArray(array, low, mid, high, ARRAYSIZE);
    }
}
int main()
{
    int array[] = {34, 54, 6, 12, 89, 0, 3}, size;
    size = sizeof(array) / sizeof(array[0]);
    TraversalArray(array, size);
    printf("\n");
    MergeSort(array, 0, size – 1, size);
    TraversalArray(array, size);
    return 0;
}
```

```
HitenRathod135@DataStructure >gcc Practical_14.c

HitenRathod135@DataStructure >a.exe
34      54      6       12      89      0       3
0       3       6       12      34      54      89
HitenRathod135@DataStructure >
```

Practical – 15

Binary search :

```
#include <stdio.h>
int main()
{
  int c, first, last, middle, n, search, array[100];

  printf("Enter number of elements\n");
  scanf("%d", &n);

  printf("Enter %d integers\n", n);

  for (c = 0; c < n; c++)
    scanf("%d", &array[c]);

  printf("Enter value to find\n");
  scanf("%d", &search);

  first = 0;
  last = n - 1;
  middle = (first+last)/2;

  while (first <= last) {
    if (array[middle] < search)
      first = middle + 1;
    else if (array[middle] == search) {
      printf("%d found at location %d.\n", search, middle+1);
      break;
```

```
    }
    else
      last = middle - 1;

    middle = (first + last)/2;
  }
  if (first > last)
    printf("Not found! %d isn't present in the list.\n", search);

  return 0;
}
```

```
HitenRathod135@DataStructure >gcc Practical_15.c

HitenRathod135@DataStructure >a.exe
Enter number of elements
5
Enter 5 integers
11
22
33
44
66
Enter value to find
44
44 found at location 4.

HitenRathod135@DataStructure >
```